

Look Before You Leap: A Self-Review Bayesian Optimization Method for Constrained High-Dimensional Design Space Exploration

Xuyang Zhao^{1†}, Yiyang Zhao^{1†}, Zheng Wu¹, Tianning Gao², Zhaori Bi^{1*}, Changhao Yan¹, Dian Zhou¹², Xuan Zeng^{1*}

¹State Key Laboratory of Integrated Chips & Systems, School of Microelectronics, Fudan University, China

²Department of Electrical Engineering, University of Texas at Dallas, U.S.A

Abstract—The parameterizable and synthesizable RISC-V processors enable the automatic generation of customized CPU cores through EDA tools. However, current methods often explore the extensive design space with significant model errors while neglecting design constraints, which are critical for practical implementations. To address these limitations, we propose a Self-Review Bayesian Optimization method (SRBO). This method integrates a teacher-student paradigm within a local Bayesian optimization framework to reduce model errors and enhance exploration efficiency. Additionally, it employs deep ensembles for effective constraint handling. Experimental results demonstrate that our approach outperforms state-of-the-art methods within a limited time budget, significantly enhancing exploration efficiency.

Index Terms—High-dimensional Design Space Exploration, Teacher-Student Paradigm, Deep Ensembles, RISC-V Processor

I. INTRODUCTION

As RISC-V processor agile design emerges and diverse application demands increase, CPU customization has become increasingly popular. Open-source frameworks, such as Chipyard [1], automatically generate Register-Transfer Level (RTL) files based on user-defined design configurations written in Chisel [2]. With the support of HAMMER framework [3] and Electronic Design Automation (EDA) tools, the entire Very Large Scale Integration (VLSI) flow, which is from Chisel to GDSII, can be automated. Designers can easily modify their designs by adjusting various design parameters, including microarchitecture parameters and EDA tool options, to achieve improved Performance, Power, and Area (PPA) outcomes. Consequently, traditional challenges of CPU design can be redefined as design parameter optimization or Design Space Exploration (DSE) problems.

However, conflicts often arise among various Quality of Results (QoRs) metrics. For instance, better performance often necessitates greater power consumption or increased area. Traditional optimization methods employ weighted approaches to balance PPAs, converting the problem into a Single-Objective Optimization (SOO) framework. Nonetheless, with the advent of hierarchical design methodologies, module specifications may need adjustment in response to upper-level design requirements. In a SOO scenario, any changes in weights assigned

to QoR metrics render the problem fundamentally different, necessitating re-optimization. On the other hand, conducting design verification for each adjustment incurs significant time costs, often requiring several weeks to complete the entire VLSI flow. Consequently, Multi-Objective Optimization (MOO) methods have been introduced into the DSE problems. MOO results in a set of Pareto solutions, each representing an optimal design that reflects a trade-off among different QoRs. When design requirements change, designers can select a design from the Pareto set according to their demands, eliminating the need for re-optimization. This significantly reduces the repeated optimizations and enhances design efficiency.

Traditional MOO methods, such as NSGA-II [4] and MOEA/D [5], are based on evolutionary algorithms. However, they require a substantial number of simulation points, which is not suitable for DSE problems that necessitate time-consuming simulation. In contrast, Bayesian optimization (BO) is a model-based optimization method that can yield effective results with relatively few actual simulation points. BOOM-Explorer [6], [7] and GRL-DSE [8] utilize BO in conjunction with features of RISC-V microarchitecture to facilitate effective exploration of the microarchitecture design space. However, these methods necessitate the evaluation of entire design space using surrogate models to identify the next data point for simulation, even when only a subset of design points requires simulation. Therefore, they consider only a limited number of microarchitecture design parameters and neglect the EDA tool parameters, which also significantly influence the design process. In contrast, REMOTune [9] and ROI-HIT [10] employ local BO [11], [12] and dimension reduction techniques [13], [14], effectively handling the high-dimensional DSE problem including the parameters of both microarchitecture and EDA tools. However, they ignore the constraint handling. In practical design scenarios, although a design yields excellent PPA results after logical synthesis, it cannot work if it violates timing constraints. Consequently, it is essential to incorporate constraints into the DSE process.

In addition, the number of parameters within the design space, encompassing both microarchitecture parameters and EDA tools (such as logic synthesis tools), can exceed 50, with the scale potentially reaching 10^{30} . Simultaneously, completing the processes of RTL generation, RTL simulation, and logic synthesis for a specific design parameter configuration

[†]Equal contribution

*Corresponding authors: {zhaori_bi, xzeng}@fudan.edu.cn

can take from 1 to 6 hours. Consequently, DSE methods can only model this vast design space based on a very limited dataset, inevitably introducing errors. Furthermore, if simulation resources are wasted on an inappropriate candidate point, it incurs significant time costs that substantially impact the effectiveness of exploration. Therefore, we propose a Self-Review Bayesian Optimization (SRBO) method for constrained high-dimensional DSE. It integrates a teacher-student paradigm within a local BO framework to reduce model errors and employs deep ensembles for effective constraint handling. Our contributions are summarized as follows:

- We propose a local Bayesian optimization based on teacher-student paradigm. To address the challenges of inaccurate modeling and low exploration efficiency within a vast design space, we concentrate on the current optimal local region and employ variable selection to identify key design parameters. Subsequently, we utilize Gaussian Process Regression (GPR) models as “students” and employ Multi-Layer Perceptron (MLP) models as “teachers” to mitigate model misdirection, thereby enhancing optimization efficiency.
- We propose a constraint classifier based on deep ensemble. To assess whether candidate design points satisfy the design constraints, we employ multiple MLPs for classification. It reduces the need for actual simulation of infeasible design points, thereby further enhancing optimization efficiency.
- The experimental results demonstrate that SRBO outperforms the state-of-the-art methods, achieving $1.93 \times \sim 41.47 \times$ hypervolume increment within the same time budget.

The remainder of this paper is structured as follows: Section II introduces the problem formulation. Section III details the SRBO algorithm. Section IV presents experimental results and compares performances of our proposed algorithm with state-of-the-art algorithms. Section V concludes this article.

II. PROBLEM FORMULATION

The DSE problem can be formulated as a constrained MOO problem, as shown in (1).

$$\begin{aligned} & \text{minimize} \quad f_1(\mathbf{x}), \dots, f_M(\mathbf{x}), \\ & \text{s.t.} \quad g(\mathbf{x}) \geq C, \end{aligned} \quad (1)$$

where $\mathbf{x}$ is design parameters and $f_i(\cdot)$ represents PPAs. $g(\cdot)$ denotes constraints, such as setup and hold time requirements.

To assess the quality of MOO, the concept of *domination* is introduced. For arbitrary data points $\mathbf{a}$ and $\mathbf{b}$, $\mathbf{a} \prec \mathbf{b}$ ($\mathbf{a}$ dominates $\mathbf{b}$) if

$$\begin{aligned} & \forall i \in \{1, \dots, M\} \quad f_i(\mathbf{a}) \leq f_i(\mathbf{b}) \\ & \text{and} \quad \exists i \in \{1, \dots, M\} \quad f_i(\mathbf{a}) < f_i(\mathbf{b}). \end{aligned} \quad (2)$$

The MOO yields a Pareto Set PS and the corresponding performance space is referred to as Pareto front PF .

$$\begin{aligned} PS &= \{\mathbf{x} \in X_C \mid \nexists \mathbf{x}' \in X_C : \mathbf{f}(\mathbf{x}') \prec \mathbf{f}(\mathbf{x})\}, \\ PF &= \{\mathbf{f}(\mathbf{x}) \mid \mathbf{x} \in PS\}, \quad X_C = \{\mathbf{x} \in X \mid g(\mathbf{x}) \geq C\}. \end{aligned} \quad (3)$$

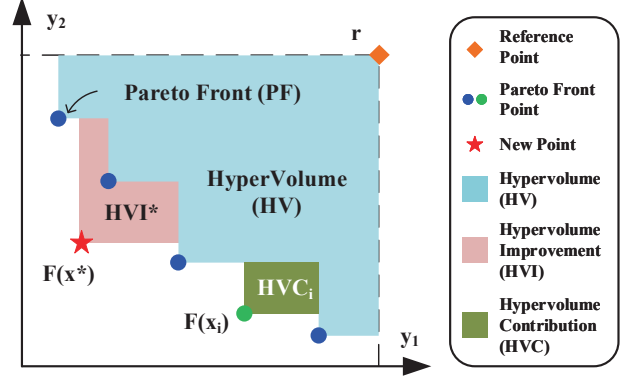


Fig. 1. The illustration of HyperVolume (HV), HyperVolume Improvement (HVI), and HyperVolume Contribution (HVC). A larger HV means a better PF.

In MOO, HyperVolume (HV) is a common performance indicator to assess the quality of the PF. As Fig. 1 shows, given a reference point $\mathbf{r}$ and a finite PF , the HV is the M -dimensional Lebesgue measure $\mathcal{M}$ of the space dominated by PF , as defined in (4).

$$HV(PF, \mathbf{r}) = \mathcal{M}\left(\bigcup_{j=1}^{|PF|} [\mathbf{r}, \mathbf{y}_j]\right), \quad (4)$$

where $[\mathbf{r}, \mathbf{y}_j]$ denotes the hyper-rectangle bounded by $\mathbf{r}$ and $\mathbf{y}_j$. A larger HV value indicates a better PF, which means a better MOO result.

To compare the points in the PS, HyperVolume Contribution (HVC) is defined as (5). Similarly, to select the best candidate point, HyperVolume Improvement (HVI) of a candidate point $\mathbf{x}^*$ is defined as (6). As shown in Fig. 1, the green area is the HVC of $\mathbf{x}_i$, and the red area is the HVI of $\mathbf{x}^*$.

$$HVC(\mathbf{x}_i | PF, \mathbf{r}) = HV(PF, \mathbf{r}) - HV(PF \setminus \mathbf{f}(\mathbf{x}_i), \mathbf{r}), \quad (5)$$

$$HVI(\mathbf{x}^* | PF, \mathbf{r}) = HV(PF \cup \mathbf{f}(\mathbf{x}^*), \mathbf{r}) - HV(PF, \mathbf{r}). \quad (6)$$

III. SELF-REVIEW BAYESIAN OPTIMIZATION

A. Overview of SRBO Framework

Our proposed SRBO framework is shown in Fig. 2. The SRBO method is built upon a local BO framework. It identifies key design parameters through a variable selection technique and explores designs near the current optimal design point with the highest HVC. The surrogate models leverage a teacher-student paradigm, where MLP models serve as the teachers and GPR models act as the students. This interaction reduces model error and yields more accurate prediction points. Additionally, SRBO employs a constraint classifier based on deep ensembles to model constraints, allowing it to assess the feasibility of candidate design points and filter out potentially invalid ones. Subsequently, the local BO computes the HVIs of the feasible candidate points based on the predicted values from the surrogate models, and the point with the largest HVI is selected for simulation. The results from the simulation will be incorporated into the explored dataset, thereby updating each model.

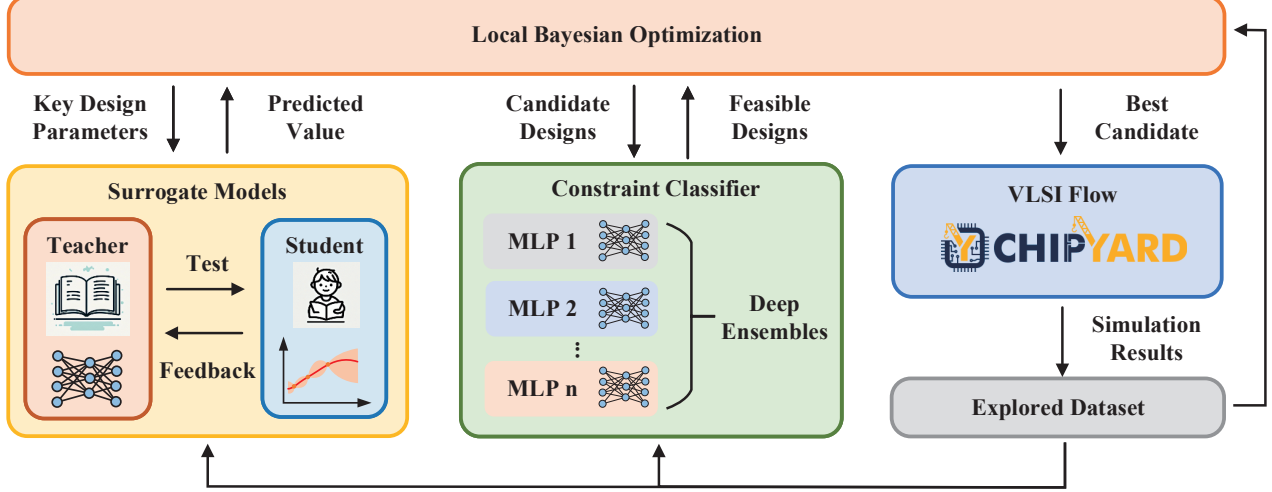


Fig. 2. An illustration of our proposed SRBO method. It utilizes a local BO framework to identify the key design variables and explore designs near the current optimal design point. The surrogate models based on the teacher-student paradigm model the PPAs in the key design space and provide the predicted value after self-review. Simultaneously, the constraint classifier based on deep ensemble assesses the constraints, filtering all candidate designs generated by the local BO to obtain the predicted feasible design points. The local BO ranks the candidates according to the predicted values provided by the surrogate models, selects the optimal candidate for simulation, and incorporates the results from the simulation into the explored dataset, updating each model with this new design point.

B. Teacher-Student Paradigm

The design space for high-dimensional DSE problems is extensive. For instance, the design space encompassing the parameters of the RISC-V BOOM microarchitecture and the Genus logical synthesis tool consists of 53 design variables with a range of 10^{30} . Consequently, modeling such a large design space using a limited number of sampled points inevitably introduces errors. These model errors may misguide the exploration process. Therefore, we employ a Teacher-Student Paradigm (TSP) to enhance model accuracy through self-review, improving overall exploration efficiency.

Traditional BO models the design space using a GPR model, which provides the mean and variance of predictions. Although the acquisition function can be employed to balance these mean and variance estimates, selecting an appropriate acquisition function and setting the hyperparameters often require sophisticated techniques, as the optimal choices can vary across different cases. This variability diminishes the robustness and generalization of BO.

TSP incorporates a teacher model and a student model. In our SRBO, the MLP model acts as the teacher $T(\cdot; \theta_T)$, and the GPR model serves as the student $S(\cdot; \theta_S)$. In addition to training with the dataset, the MLP incorporates the error between the predictions of the GPR model and the true outcomes into its loss function. By leveraging the GPR model's strong expressiveness while mitigating its misleading uncertainties, the MLP achieves more accurate predictions through a process of self-review, eliminating the need for experienced tricks.

As the Fig. 3 shows, firstly, the teacher (the MLP model) predicts the values of the candidate points (the red stars) based on the explored points D (the blue points).

$$\hat{\mathbf{y}}_{cand}(\theta_T) = T(\mathbf{X}_{cand}; \theta_T), \quad (7)$$

where $\mathbf{X}_{cand}$ denote the candidate points.

Then, the teacher provides the candidate points and a selection of the explored points to the student for modeling.

$$\theta_S^*(\theta_T) = \arg \min_{\theta_S} L_S(D_{test}(\theta_T); \theta_S), \quad (8)$$

$$D_{test}(\theta_T) = \{\mathbf{X}_{cand} \cup \mathbf{X}_{valid}, \hat{\mathbf{y}}_{cand}(\theta_T) \cup \mathbf{y}_{valid}\}, \quad (9)$$

where $\{\mathbf{X}_{valid}, \mathbf{y}_{valid}\}$ are extracted from the explored dataset D . L_S represent the loss function of the student. The student evaluates the explored points and returns the error between the predicted values and the actual values to the teacher.

$$L_{test}(D; \theta_S^*(\theta_T)) = MSE(\mu_{\theta_S^*(\theta_T)}(\mathbf{X}), \mathbf{y}). \quad (10)$$

After that, the teacher is retrained in accordance with the loss function of the teacher L_T and the loss function L_{test} derived from the student's feedback to develop a more accurate model.

$$\theta_T^* = \arg \min_{\theta_T} L_T(D; \theta_T) + \lambda L_{test}(D; \theta_S^*(\theta_T)). \quad (11)$$

In this work, our teacher model is set up as follows:

$$\begin{aligned} T(\mathbf{x}; \theta_T) : & \text{Input } \mathbf{x} \rightarrow \text{Linear}(n, 256) \rightarrow \text{ReLU} \\ & \rightarrow \text{Linear}(256, 64) \rightarrow \text{ReLU} \\ & \rightarrow \text{Linear}(64, 32) \rightarrow \text{ReLU} \\ & \rightarrow \text{Linear}(32, 1) \rightarrow \text{Output } y \end{aligned} \quad (12)$$

Acknowledging the potential for misleading pseudo-label predictions by the teacher, the student is evaluated against the existing ground truth labeled data, and its performance is consequently communicated back to the teacher. Upon the completion of the teacher-student interaction, the teacher, refined through regularization, produces reliable pseudo-labels for training the GPR model, thereby enhancing its data querying capability, even with limited labeled training data.

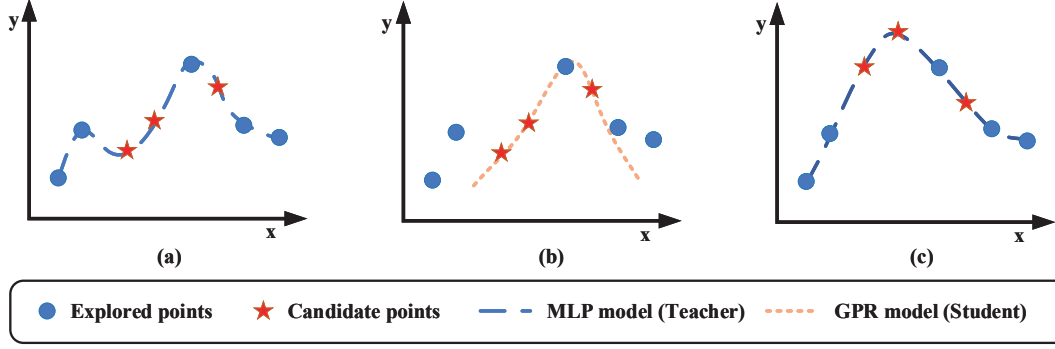


Fig. 3. An illustration of the Teacher-Student Paradigm (TSP). The blue points represent explored points, while the red stars denote candidate points. In the TSP of SRBO, the MLP model acts as the teacher, and the GPR model serves as the student. (a) Initially, the teacher predicts the values of the candidate points based on the explored points. (b) Subsequently, the teacher provides the candidate points and a selection of the explored points to the student for modeling. The student evaluates these explored points and returns the error between the predicted values and the actual values to the teacher, which is utilized as a loss function. (c) The teacher is then retrained in accordance with the loss function derived from the student’s feedback to develop a more accurate model.

C. Constraint Classifier based on Deep Ensembles

In RISC-V processor design, it is crucial to adhere to the specified design constraints. For instance, if the timing constraints are not satisfied, even a design with optimal PPAs may result in functional errors, rendering it infeasible. Existing DSE methods often overlook constraint handling and typically only investigate design spaces that comply with these constraints. However, in the vast, high-dimensional DSE, there may exist design points that do not satisfy timing constraints, necessitating effective constraint management.

A common approach for handling constraints is to penalize design points that deviate from the required constraints. However, in digital IC design, a lack of adherence to constraints like timing can result in substantial changes across the entire design. At times, fine-tuning parameters may fail to yield a design that meets these constraints, rendering the distance from the constraint requirements ineffective as a guide for exploration direction.

To address these challenges, we propose using deep ensembles to transform the satisfaction of design constraints into classification problems. This involves utilizing multiple MLPs as classifiers and aggregating their predicted outputs to derive final results.

Deep Ensembles (DE) consists of a collection of neural networks, commonly MLPs, and are employed to model epistemic uncertainty. By training multiple independent MLPs and harnessing their diversity, DE improves accuracy and enhances uncertainty estimation. Each MLP in the ensemble is trained on the same dataset but initialized with different random weights. During inference, predictions from the ensemble are derived by averaging or combining the outputs of the individual models.

In the SRBO, we independently train N identical MLPs on all explored points D to model the probability of each design being either feasible or infeasible. The output of each MLP is a probability value between 0 and 1 for a given design point x . Our constraint classifier computes the mean of the predicted

values, utilizing the trained weights θ_i^* from each model.

$$C(x) = \frac{1}{N} \sum_i^N M_{\theta_i^*}(x). \quad (13)$$

In this work, we propose that if $C(x) > 0.5$, x is considered a feasible design candidate for subsequent optimization. We utilized 5 MLPs to construct the DE, where each MLP comprises 2 hidden layers, each containing 64 neurons.

D. Local Bayesian Optimization

In addition to the vast design space associated with high-dimensional DSE problems, the time required for design simulation and verification is substantial. For instance, generating RTL files, performing RTL simulation, and executing logic synthesis for a single RISC-V BOOM design can take between 1 and 6 hours. Consequently, achieving a global optimum within a limited time budget is impractical. In this context, identifying a better solution is often more valuable than pursuing the absolute best. Therefore, our approach prioritizes the exploration of current optimal design points in the vicinity and emphasizes key design parameters, which significantly reduces the problem’s dimensionality and enhances optimization efficiency.

Like ROI-HIT [10], we apply a sensitivity matrix S [15] to measure the importance of variables.

$$S = \{s_{ij} \mid i = 1, \dots, n, j = 1, \dots, M\}, \quad (14)$$

$$s_{ij} = \frac{\text{Var}[E(y_j|x_i)]}{\text{Var}(y_j)} \approx \frac{\sum_{l=1}^{r_i} |A_i^l| [(\bar{y}_j)_i^l - \bar{y}_j]^2}{\sum_{x \in X} [y_j(x) - \bar{y}_j]^2}, \quad (15)$$

$$\bar{y}_j = \frac{1}{|X|} \sum_{x \in X} y_j(x), \quad (\bar{y}_j)_i^l = \frac{1}{|A_i^l|} \sum_{x \in A_i^l} y_j(x),$$

where X represents the dataset, r_i represents the number of value which x_i can be and A_i^l represents the set of points for which the variable x_i is set to its l -th value.

Given that the current optimal point in our work satisfies the constraints, we posit that the key design parameters should exhibit sensitivity to PPAs while remaining insensitive to the

constraints. Consequently, we assign equal weights to PPAs and assign negative values to the weights of the constraints. The key design variables are obtained as follows.

$$V_i = \sum_{j=1}^m (w_j \cdot s_{ij}) + C_p \sqrt{1/n_i}, \quad (16)$$

$$var_{key} = \{var_i \mid V_i \geq p \cdot V_{max}\}, \quad (17)$$

where n_i is the number that the i -th variable has been selected as the key variable, and C_p and p are hyper-parameters. In this work, $C_p = 0.01$ and $p = 0.5$.

We select the explored point with the highest HVC, which can be calculated using (5), as the optimal design. The design space is then restricted to the vicinity of this current optimum. For non-key variables, the values remain consistent with the current optimum, while for the key variables, only the values within the neighboring interval are considered as candidate points. Surrogate models and constraint classifier are trained exclusively for the key variables. Candidate points are firstly filtered from infeasible points using the constraint classifier. Subsequently, surrogate models are employed to predict their PPAs, followed by the calculation of HVI using (6). The design exhibiting the maximum HVI will be simulated.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

We explore the design space for a RISC-V BOOM [16], [17] core and a Rocket [18] core, both configured as RV64GC. The PPA results are evaluated using System-on-Chip (SoC) designs in all experiments, each equipped with either the BOOM core or the Rocket core as its processor. The peripheral modules of these SoCs consist of standard components provided by the Chipyard framework. A 7nm ASAP7 Process Design Kit (PDK) [19] is utilized within the VLSI flow. Since ASAP7-based designs are not manufacturable, this paper focuses solely on the front-end design for the experiments. The design parameters are shown in TABLE I.

Firstly, the RTL design is generated by Chipyard framework. Then, an open-source simulator Verilator [20] simulates the design to obtain Cycle Per Instruction (CPI). 13 benchmarks are selected to evaluate the performance of the design: *dhrystone*, *median*, *mm*, *mt-matmul*, *mt-vvadd*, *multiply*, *qsort*, *rsort*, *spmv*, *towers*, *vvadd*, *fir2dim*, and *whetstone*. The average CPI of benchmarks is taken as the performance metric. Finally, Cadence Genus Synthesis Solution 19.10 is used as the logic synthesis tool to report the timing violations and estimate power and area metrics. The power metric is the total power. The clock frequency is set as 500MHz. Design points that fail to meet the timing constraints are categorized as failure points and are excluded from the resulting statistics.

We compare SRBO with 4 state-of-the-art methods: (1) BOOM-Explorer [6], a BO-based method for microarchitecture DSE, (2) GRL-DSE [8], another BO method for microarchitecture DSE, (3) REMOTune [9], a local BO method for high-dimensional DSE that by the trust regions and random embedding, (4) ROI-HIT [10], a recent local BO method for

TABLE I
DESIGN PARAMETERS

Class	N	Name	Value
BOOM	22	BPD	Boom2,TAGEL,Alpha21264
		fetchWidth	4,8
		numFetchBufferEntries	8,16,24,32,35,40
		numRasEntries	16,24,32
		maxBrCount	8,12,16,20
		decodeWidth	1,2,3,4,5
		numRobEntries	32,64,96,128,130
		numIntPhysRegisters	48,64,80,96,112
		memIssueWidth	1,2
		intIssueWidth	1,2,3,4,5
		numLdqEntries	8,16,24,32
		enablePrefetching	false,true
		enableSFBOpt	false,true
		numRXQEntries	2,4,8
		numRCQEntries	8,16,32
		nL2TLBEntries	128,256,512,1024
		nL2TLBWays	1,2,4,8
		nLCacheWays	2,4,8
		nLCacheTLBWays	8,16,32
		nDCacheWays	2,4,8
		nDCacheMSHRs	2,4,8
		nDCacheTLBWays	8,16,32
Rocket	9	mulUnroll	1,2,4,8
		divUnroll	1,2,4,8
		mulEarlyOut	false,true
		divEarlyOut	false,true
		nDCacheWays	1,2,4
		nDCacheTLBWays	4,8,16
		nDCacheMSHRs	1,2,4
		nLCacheWays	1,2,4
		nLCacheTLBWays	4,8,16
Genus	31	time_recovery_arcs	false,true
		auto_partition	true,false
		dp_analytical_opt	off,standard,extreme
		dp_area_mode	false,true
		dp_csa	basic,none
		dp_rewriting	basic,advanced,none
		dp_sharing	advanced,basic,none
		exact_match_seq_async_ctrls	false,true
		exact_match_seq_sync_ctrls	false,true
		iopt_enable_floating_output_check	false,true
		iopt_force_constant_removal	false,true
		iopt_lp_power_analysis_effort	low,medium,high
		lbr_seq_in_out_phase_opto	false,true
		optimize_net_area	true,false
		retime_effort_level	low,medium,high
		retime_optimize_reset	false,true
		syn_generic_effort	low,medium,high
		syn_map_effort	low,medium,high
		syn_opt_effort	low,medium,high
		leakage_power_effort	low,medium,high
		lp_power_analysis_effort	low,medium,high
		enable_latch_max_borrow	false,true
		latch_max_borrow	1:500:1
		enable_max_fanout	false,true
		max_fanout	2:64:1
		enable_lp_clock_gating_max_flops	false,true
		lp_clock_gating_max_flops	8,16,32
		enable_lp_power_optimization_weight	false,true
		lp_power_optimization_weight	0.1:0.9:0.1
		max_dynamic_power	3.0:150.0:0.1
		max_leakage_power	5.0:100.0:0.1

high-dimensional DSE that combines the ROIs and variable selection. To verify the effectiveness of TSP and DE, we implement two versions of SRBO: (1) SRBO w/o TSP, a SRBO method without TSP, (2) SRBO, the full version of SRBO.

All algorithms are implemented in Python. All algorithms are initialized with the same set of 100 designs. The experiments are executed within a maximum time budget of one day (86400 seconds). The HV indicator serves as a benchmark

TABLE II
OPTIMIZATION RESULTS FOR RISC-V BOOM DSE

Algorithm	Δ HV	Ratio	# Fail.	# Sim.	Rate
BOOM-Explorer [6]	2	0.00×	0	12	0.00%
GRL-DSE [8]	726	0.00×	2	9	22.22%
REMOTune [9]	21,597	1.00×	6	13	46.15%
ROI-HIT [10]	295,011	13.66×	4	16	25.00%
SRBO w/o TSP	577,879	26.76×	3	16	18.75%
SRBO	895,622	41.47×	3	17	17.65%

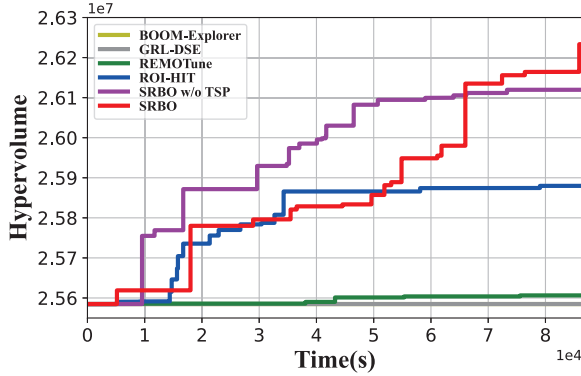


Fig. 4. The optimization results for RISC-V BOOM DSE.

metric for evaluating the quality of the solutions generated by the algorithms. For HV computation, the reference point is defined as the maximum observed values of CPI, power, and area within the initial dataset. The experiments are conducted five times, and the average results are subsequently reported.

B. RISC-V BOOM Design Space Exploration

The optimization results are shown in TABLE II and Fig. 4. In the table, Δ HV denote the increase in HV compared to the HV of the initial dataset, while # Fail. and # Sim. represent the number of failure points and simulation points, respectively. Compared with the state-of-the-art methods, SRBO obtains the largest Δ HV and achieves $41.47\times$ and $3.04\times$ HV increment over REMOTune and ROI-HIT.

BOOM-Explorer and GRL-DSE are well-suited for microarchitecture design space exploration (DSE) problems with relatively few design variables. However, for high-dimensional DSE problems characterized by extensive design spaces, both BOOM-Explorer and GRL-DSE exhibit less increase in HV. The introduction of the local BO method significantly enhances the performance of ROI-HIT, SRBO w/o TSP, and SRBO during the early stages of optimization. Additionally, the implementation of a constraint classifier allows SRBO w/o TSP and SRBO to achieve a lower failure rate. Although the BOOM-Explorer has the zero failure rate, it has little HV increase, so its failure rate has little significance. Furthermore, thanks to the teacher-student paradigm, the accuracy of the SRBO model continually improves, enabling it to escape from local optima in the ROI-HIT method.

C. Rocket Design Space Exploration

The optimization results are shown in TABLE III and Fig. 5. Compared with the state-of-the-art methods, SRBO obtains the

TABLE III
OPTIMIZATION RESULTS FOR ROCKET DSE

Algorithm	Δ HV	Ratio	# Fail.	# Sim.	Rate
BOOM-Explorer [6]	65	0.00×	0	16	0.00%
GRL-DSE [8]	1,741	0.11×	5	24	20.83%
REMOTune [9]	15,822	1.00×	1	29	3.45%
ROI-HIT [10]	55,272	3.49×	2	29	6.90%
SRBO w/o TSP	72,618	4.59×	0	27	0.00%
SRBO	107,470	6.79×	0	26	0.00%

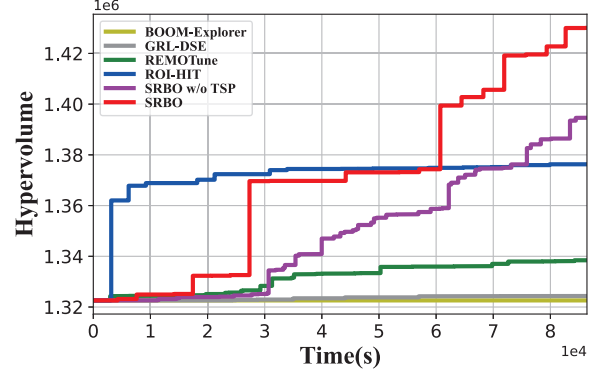


Fig. 5. The optimization results for Rocket DSE.

largest Δ HV and achieves $6.79\times$ and $1.94\times$ HV increment over REMOTune and ROI-HIT.

The optimization results for Rocket are generally consistent with those of RISC-V BOOM. However, Rocket is relatively easier to optimize within the specified constraints, allowing other unconstrained methods to also achieve favorable optimization results. As illustrated in the Fig. 5, both SRBO and ROI-HIT became trapped in the same local optimum region. But, due to the self-review model, SRBO successfully escapes from this region.

V. CONCLUSION

In this paper, we propose a self-review Bayesian optimization method for constrained high-dimensional DSE. To address the challenges posed by the vast design space and time-consuming simulations, SRBO integrates a teacher-student paradigm within a local BO framework to reduce model errors and enhance the exploration efficiency. Given the importance of design constraints in digital IC designs, SRBO utilizes deep ensembles for effective constraint handling. Experimental results demonstrate that our approach outperforms state-of-the-art methods within a limited time budget. Our proposed SRBO method is a general framework that we anticipate will be applicable to other high-dimensional, expensive optimization problems with constraints in the future.

ACKNOWLEDGMENTS

This research is supported partly by National Key R&D Program of China 2020YFA0711900, 2020YFA0711901, partly by National Natural Science Foundation of China (NSFC) research projects 92473206, 62304052, 62474050, 92473207, 62474051.

REFERENCES

- [1] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton *et al.*, “Chipyard: Integrated design, simulation, and implementation framework for custom socs,” *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.
- [2] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzyniek, and K. Asanović, “Chisel: Constructing hardware in a scala embedded language,” in *2012 Design Automation Conference (DAC)*, 2012, pp. 1212–1221.
- [3] H. Liew, D. Grubb, J. Wright, C. Schmidt, N. Krzysztofowicz, A. Izraelevitz, E. Wang, K. Asanović, J. Bachrach, and B. Nikolić, “Hammer: a modular and reusable physical design flow tool,” in *2022 59th ACM/IEEE Design Automation Conference (DAC)*, 2022, pp. 1335–1338.
- [4] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: Nsga-ii,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [5] Q. Zhang and H. Li, “Moea/d: A multiobjective evolutionary algorithm based on decomposition,” *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, pp. 712–731, 2007.
- [6] C. Bai, Q. Sun, J. Zhai, Y. Ma, B. Yu, and M. D. Wong, “Boom-explorer: Risc-v boom microarchitecture design space exploration framework,” in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2021, pp. 1–9.
- [7] —, “Boom-explorer: Risc-v boom microarchitecture design space exploration,” *ACM Transactions on Design Automation of Electronic Systems*, 2023.
- [8] X. Yi, J. Lu, X. Xiong, D. Xu, L. Shang, and F. Yang, “Graph representation learning for microarchitecture design space exploration,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [9] S. Zheng, H. Geng, C. Bai, B. Yu, and M. D. Wong, “Boosting vlsi design flow parameter tuning with random embedding and multi-objective trust-region bayesian optimization,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 28, no. 5, pp. 1–23, 2023.
- [10] X. Zhao, T. Gao, A. Zhao, Z. Bi, C. Yan, F. Yang, S.-G. Wang, D. Zhou, and X. Zeng, “Roi-hit: Region of interest-driven high-dimensional microarchitecture design space exploration,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 11, pp. 4178–4189, 2024.
- [11] D. Eriksson, M. Pearce, J. Gardner, R. D. Turner, and M. Poloczek, “Scalable global optimization via local bayesian optimization,” *Advances in neural information processing systems (NeurIPS)*, vol. 32, 2019.
- [12] S. Daulton, D. Eriksson, M. Balandat, and E. Bakshy, “Multi-objective bayesian optimization over high-dimensional search spaces,” in *Uncertainty in Artificial Intelligence (UAI)*. PMLR, 2022, pp. 507–517.
- [13] L. Song, K. Xue, X. Huang, and C. Qian, “Monte carlo tree search based variable selection for high dimensional bayesian optimization,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 28 488–28 501, 2022.
- [14] Z. Wang, F. Hutter, M. Zoghi, D. Matheson, and N. De Freitas, “Bayesian optimization in a billion dimensions via random embeddings,” *Journal of Artificial Intelligence Research*, vol. 55, pp. 361–387, 2016.
- [15] L. Adjengue, C. Audet, and I. Ben Yahia, “A variance-based method to rank input variables of the mesh adaptive direct search algorithm,” *Optimization Letters*, vol. 8, pp. 1599–1610, 2014.
- [16] C. Celio, D. A. Patterson, and K. Asanovic, “The berkeley out-of-order machine (boom): An industry-competitive, synthesizable, parameterized risc-v processor,” *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2015-167*, 2015.
- [17] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanovic, “Sonicboom: The 3rd generation berkeley out-of-order machine,” in *Fourth Workshop on Computer Architecture Research with RISC-V*, vol. 5, 2020, pp. 1–7.
- [18] K. Asanovic, R. Avižienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Dabbelt, J. Hauser, A. Izraelevitz *et al.*, “The rocket chip generator,” *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17*, vol. 4, pp. 6–2, 2016.
- [19] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, “Asap7: A 7-nm finfet predictive process design kit,” *Microelectronics Journal*, vol. 53, pp. 105–115, 2016.
- [20] W. Snyder, “Verilator 4.0: open simulation goes multithreaded,” in *Open Source Digital Design Conference (ORConf)*, 2018.